SQL performance and database operations

Read query plans, add the right indexes, untangle locks, size connection pools and run replication, vacuum and day-two operations.

A MARGA printable pack. Free, offline, no sign-up. Learn online at marga.eknathalabs.com.

MARGA by EknathaLabs
marga.eknathalabs.com

Contents

1. Reading EXPLAIN and finding slow queries (15 min)
2. Indexes that help and indexes that hurt (14 min)
3. Locks, deadlocks and long transactions (15 min)
4. Connections, pooling and capacity (14 min)
5. Replication lag, vacuum and day-two operations (15 min)
6. Finding the worst queries with pg_stat_statements (13 min)
7. Partitioning and data retention (12 min)
8. Backup and point-in-time recovery operations for SQL (13 min)
9. Online schema changes without downtime (13 min)
10. Replicas, switchover and failover operations (13 min)
11. Autovacuum, bloat and maintenance windows (12 min)
12. A database incident runbook (12 min)
Cheat sheet

Lesson 1. Reading EXPLAIN and finding slow queries

15 minutes. Find the queries that cost the most, read an EXPLAIN plan, and tell a missing index from stale statistics.

Most database slowness comes from a small number of queries. The work is to find them, read what the database does with them, and change one thing at a time. The examples use PostgreSQL, and the ideas carry over to MySQL and others.

What you will learn

- Finding expensive queries with pg_stat_statements and the slow query log
- Reading EXPLAIN and EXPLAIN ANALYZE
- Sequential scan, index scan and join types
- What stale statistics look like

Find the queries first

Enable the `pg_stat_statements` extension. It records every normalized query with how many times it ran and how long it took.

```
$ psql -c "SELECT calls, round(total_exec_time::numeric,0) AS total_ms, round(mean_exec_time::nume \
    ric,1) AS mean_ms, left(query,60) AS query FROM pg_stat_statements ORDER BY total_exec_time DE \
    SC LIMIT 3;"
 calls  | total_ms | mean_ms | query
--------+----------+---------+------------------------------------------------------------
 480211 |  9120432 |    19.0 | SELECT * FROM orders WHERE customer_id = $1 ORDER BY
   1210 |  4411200 |  3645.5 | SELECT customer_id, sum(total) FROM orders WHERE cre
```

Sort by **total** time, not by the slowest single run. A 19 ms query that runs 480 000 times costs more than a 3.6 s report that runs 1 200 times. The setting `log_min_duration_statement = 500` writes any statement slower than 500 ms to the log, which catches queries your extension view has not reached yet.

EXPLAIN

`EXPLAIN` shows the plan the database chooses. `EXPLAIN (ANALYZE, BUFFERS)` runs the query and shows what really happened. Do not use ANALYZE on a statement that changes data without a transaction you roll back.

```
$ psql -c "EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM orders WHERE customer_id = 4711;"
 Seq Scan on orders  (cost=0.00..412345.00 rows=11 width=96) (actual time=0.031..1893.2 rows=9 loo \
    ps=1)
   Filter: (customer_id = 4711)
   Rows Removed by Filter: 9999991
   Buffers: shared hit=1204 read=211141
 Planning Time: 0.12 ms
 Execution Time: 1893.3 ms
```

Read it from the inside out and look for:

- **Seq Scan** on a big table that returns a few rows. The database read ten million rows to find nine (`Rows Removed by Filter`). An index on `customer_id` would let it jump to them.
- **Index Scan / Index Only Scan / Bitmap Heap Scan**: the database uses an index. Fewer buffers read is the goal.
- **Estimated rows vs actual rows**: `rows=11` versus `rows=9` is fine. When the estimate is 10 and the actual is 2 000 000, the planner made its choice with wrong information. Run `ANALYZE table;` to refresh statistics.
- **Sort** with `Sort Method: external merge Disk` means the sort spilled to disk because `work_mem` was too small for that query.
- **Nested Loop** is good for few rows and terrible for many. **Hash Join** and **Merge Join** suit larger sets.

MARGA by EknathaLabs
marga.eknathalabs.com

Change one thing

After a change, run the same EXPLAIN again and compare the time and the buffers. Keep the old plan text in your ticket, so a reviewer can see the difference.

```
$ psql -c "CREATE INDEX CONCURRENTLY idx_orders_customer ON orders (customer_id);"
CREATE INDEX
$ psql -c "EXPLAIN (ANALYZE) SELECT * FROM orders WHERE customer_id = 4711;" | grep -E "Index|Exec \
    ution"
 Index Scan using idx_orders_customer on orders  (actual time=0.040..0.061 rows=9 loops=1)
 Execution Time: 0.09 ms
```

`CONCURRENTLY` builds the index without blocking writes. It takes longer and cannot run inside a transaction block.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Reading EXPLAIN and finding slow queries fits together

Part	What it does	What goes wrong
Find the queries first	Enable the pg_stat_statements extension.	Tuning the slowest single query and ignoring the one that runs a million times.
EXPLAIN	EXPLAIN shows the plan the database chooses.	Running EXPLAIN ANALYZE on writes in production without a rollback.
Change one thing	After a change, run the same EXPLAIN again and compare the time and the buffers.	Adding indexes before reading the plan.

Flow: Find the queries first to EXPLAIN / EXPLAIN to Change one thing

Incident: The query that was fast yesterday

Incident: *A checkout query went from 5 ms to 4 seconds overnight with no code change.*

```
$ psql -c "EXPLAIN (ANALYZE) SELECT * FROM carts WHERE status='open' AND user_id=88;"
 Nested Loop  (cost=0.85..120.40 rows=3 width=64) (actual time=0.03..4012.7 rows=118002 loops=1)
$ psql -c "SELECT relname, last_autoanalyze, n_live_tup FROM pg_stat_user_tables WHERE relname='ca \
    rts';"
 carts | 2026-10-19 02:11:04 | 3120
```

The planner expected 3 rows and found 118 002. The table had been loaded with a large import the night before, and statistics were two weeks old, so the planner chose a nested loop that suited a small table. `ANALYZE carts;` fixed the plan in seconds. The lesson is to compare estimated and actual rows before adding indexes: a missing index and stale statistics look similar and have different cures.

Common mistakes

- Tuning the slowest single query and ignoring the one that runs a million times
- Adding indexes before reading the plan
- Running EXPLAIN ANALYZE on writes in production without a rollback
- Judging by a query on a small development database

Check yourself

Which view lets you rank queries by total time spent?

Answer: pg_stat_statements

What does Rows Removed by Filter with a huge number tell you?

Answer: The database read many rows to find a few, so an index may help

The planner expects 3 rows but 118 000 are returned. What should you try first?

MARGA by Eknatha Labs
marga.eknathalabs.com

Answer: Run ANALYZE to refresh statistics

Why use CREATE INDEX CONCURRENTLY on a busy table?

Answer: It builds the index without blocking writes

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 2. Indexes that help and indexes that hurt

14 minutes. Design composite, partial and covering indexes, and find unused or duplicate ones that slow writes.

An index is a trade. It makes some reads fast and makes every write slower and larger. Good index design means adding the few that matter and removing the ones that do not.

What you will learn

- How a B-tree index is used, and why column order matters
- Composite, partial and covering indexes
- The write cost of indexes and how to find unused ones
- Why an index is sometimes ignored

How a B-tree works for you

A B-tree index keeps values sorted. It answers equality, ranges and `ORDER BY` on the indexed columns. With a **composite** index on `(customer_id, created_at)`, rows are sorted by customer first, then by date inside each customer.

The rule of thumb for column order: columns tested for **equality** first, then the column used for a **range or sort**.

```
$ psql -c "CREATE INDEX CONCURRENTLY idx_orders_cust_created ON orders (customer_id, created_at);"
$ psql -c "EXPLAIN SELECT * FROM orders WHERE customer_id = 4711 AND created_at >= '2026-10-01' OR \
    DER BY created_at DESC LIMIT 20;" | head -3
 Limit  (cost=0.57..38.10 rows=20 width=96)
   ->  Index Scan Backward using idx_orders_cust_created on orders
         Index Cond: ((customer_id = 4711) AND (created_at >= '2026-10-01'))
```

This one index serves queries by customer alone, and by customer plus date, and also gives the sort for free. It does **not** help a query that filters only on `created_at`, because the sorted order begins with customer.

Partial and covering indexes

A **partial index** covers only some rows. If 2 percent of orders are `status='open'` and queries look only for those, a smaller index is faster to scan and cheaper to maintain.

```
CREATE INDEX CONCURRENTLY idx_orders_open ON orders (customer_id) WHERE status = 'open';
```

A **covering index** adds extra columns so the database can answer from the index alone (an index-only scan):

```
CREATE INDEX CONCURRENTLY idx_orders_cust_total ON orders (customer_id) INCLUDE (total);
```

Index-only scans also need the table's visibility map to be current, which vacuum keeps up to date.

The cost of indexes

Every insert updates every index on the table. Every update that changes an indexed column does too. Indexes also take disk and memory and slow vacuum. A table with twelve indexes may have write speed far below one with three.

Find indexes that are never used:

```
$ psql -c "SELECT relname, indexrelname, idx_scan, pg_size_pretty(pg_relation_size(indexrelid)) AS \
     size FROM pg_stat_user_indexes ORDER BY idx_scan, pg_relation_size(indexrelid) DESC LIMIT 3;"
 relname | indexrelname        | idx_scan | size
---------+---------------------+----------+--------
 orders  | idx_orders_status   |        0 | 1.8 GB
 orders  | idx_orders_created  |       12 | 1.6 GB
```

MARGA by EknathaLabs marga.eknathalabs.com

`idx_scan = 0` since the last statistics reset is a candidate for removal, but check first that the statistics cover a full business cycle (month end, reporting jobs) and that the index does not enforce a unique constraint. Drop with `DROP INDEX CONCURRENTLY`.

Duplicate indexes are common too: an index on `(customer_id)` is redundant when `(customer_id, created_at)` exists, because the composite serves the same lookups.

When the planner ignores an index

- The query returns a large part of the table, so reading the table directly is cheaper.
- A function wraps the column: `WHERE lower(email) = 'a@b.c'` cannot use an index on `email`. Create an expression index on `lower(email)`, or change the query.
- The types differ and force a cast of the column.
- Statistics are stale and the planner thinks the table is small.
- A leading wildcard: `LIKE '%smith'` cannot use a plain B-tree.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Indexes that help and indexes that hurt fits together

Part	What it does	What goes wrong
How a B-tree works for you	A B-tree index keeps values sorted. It answers equality, ranges and ORDER BY on the indexed columns.	Putting the range column before the equality column in a composite index.
Partial and covering indexes	A partial index covers only some rows. If 2 percent of orders are status='open' and queries look only for those, a smaller index is faster to scan and cheaper to...	Adding an index for every slow query without checking existing ones.
The cost of indexes	Every insert updates every index on the table.	Dropping an index that backs a unique or foreign key constraint.
When the planner ignores an index	The query returns a large part of the table, so reading the table directly is cheaper.	Wrapping an indexed column in a function.

Flow: How a B-tree works for you to Partial and covering indexes / Partial and covering indexes to The cost of indexes / The cost of indexes to When the planner ignores an index

Incident: Twenty indexes and a slow import

Incident: *A nightly import that took 20 minutes grew to 5 hours over a year.*

```
$ psql -c "SELECT count(*) FROM pg_indexes WHERE tablename='events';"
 count
-------
    23
$ psql -c "SELECT pg_size_pretty(pg_table_size('events')) AS table, pg_size_pretty(pg_indexes_size \
    ('events')) AS indexes;"
 table  | indexes
--------+---------
 41 GB  | 87 GB
```

Over time people added an index for each new report. The import had to update 23 indexes for every row, and the indexes were twice the size of the table. A review found nine unused and four duplicate indexes. Removing thirteen brought the import back under an hour. Each index had been a reasonable choice when it was added; nobody owned the total.

Common mistakes

- Adding an index for every slow query without checking existing ones

MARGA by EknathaLabs marga.eknathalabs.com

- Putting the range column before the equality column in a composite index
- Wrapping an indexed column in a function
- Dropping an index that backs a unique or foreign key constraint

Check yourself

For a query WHERE customer_id = 5 AND created_at > X ORDER BY created_at, which index order is best?

Answer: (customer_id, created_at)

What does a partial index do?

Answer: Indexes only rows that match a condition

Why can WHERE lower(email) = 'a@b.c' ignore an index on email?

Answer: The function changes the value, so the index order cannot be used

What is the main cost of having many indexes?

Answer: Every write must update all of them

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 3. Locks, deadlocks and long transactions

15 minutes. Find what is blocking what, understand idle-in-transaction sessions and deadlocks, and set timeouts that protect the database.

A query can be slow because it is working, or because it is waiting. Locks are the usual reason for waiting, and one forgotten transaction can stop a whole application.

What you will learn

- Row locks, table locks and how they conflict
- Finding the blocker with pg_stat_activity and pg_blocking_pids
- Idle in transaction and why it is dangerous
- Deadlocks and the timeouts that limit damage

Locks in short

When a transaction updates a row, it holds a lock on it until the transaction ends. Other writers to that row wait. Readers are not blocked, because PostgreSQL keeps old row versions for them (MVCC). Statements such as `ALTER TABLE` take stronger table-level locks, and those can queue behind a long reader and then block everyone behind them.

The key sentence: **locks are held until COMMIT or ROLLBACK**, not until the statement ends.

Find the blocker

```
$ psql -c "SELECT pid, state, wait_event_type, wait_event, now()-xact_start AS tx_age, left(query, \
    50) AS query FROM pg_stat_activity WHERE datname='shop' ORDER BY xact_start;"
 pid  | state               | wait_event_type | wait_event    | tx_age   | query
------+---------------------+-----------------+---------------+----------+----------------------- \
    ------------------------
 2210 | idle in transaction |                 |               | 03:12:44 | UPDATE accounts SET bal \
    ance = balance - 10 WHERE
 2488 | active              | Lock            | transactionid | 00:41:02 | UPDATE accounts SET bal \
    ance = balance + 10 WHERE
$ psql -c "SELECT pid, pg_blocking_pids(pid) AS blocked_by FROM pg_stat_activity WHERE cardinality \
    (pg_blocking_pids(pid)) > 0;"
 pid  | blocked_by
------+------------
 2488 | {2210}
```

Process 2488 waits for a lock held by 2210. Process 2210 is **idle in transaction**: it started a transaction, did an update, and then did nothing for three hours. The client opened a transaction and never finished it, often because of a bug, a crashed worker, or someone's session left open in a terminal.

You can stop the blocker with `SELECT pg_terminate_backend(2210);`. That rolls back its work, so ask who owns it if the data matters. `pg_cancel_backend(pid)` cancels only the running statement and is gentler.

Long transactions hurt in other ways

A transaction that stays open holds back the cleanup of old row versions across the database, so tables grow and vacuum cannot remove dead rows. It also holds locks and may keep a replica in conflict. Keep transactions short, and never keep one open while waiting for a person or a slow outside call.

Deadlocks

A **deadlock** happens when two transactions each hold a lock the other needs. PostgreSQL detects it after `deadlock_timeout` (1 second by default), aborts one of them with an error, and lets the other continue.

```
ERROR:  deadlock detected
DETAIL:  Process 2210 waits for ShareLock on transaction 8841; blocked by process 2488.
         Process 2488 waits for ShareLock on transaction 8839; blocked by process 2210.
HINT:  See server log for query details.
```

MARGA by EknathaLabs
marga.eknathalabs.com

The cure is in the application: touch rows in the same order everywhere (for example, always update the account with the lower id first), keep transactions short, and retry the failed transaction.

Timeouts that protect you

- `lock_timeout`: how long a statement waits for a lock before giving up. Set it for migrations, so an `ALTER TABLE` that cannot get its lock fails quickly instead of queueing behind a reader and freezing the table.
- `idle_in_transaction_session_timeout`: ends sessions that sit in a transaction doing nothing.
- `statement_timeout`: caps how long a single statement may run.

```
ALTER ROLE app SET idle_in_transaction_session_timeout = '60s';
ALTER ROLE app SET statement_timeout = '30s';
```

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Locks, deadlocks and long transactions fits together

Part	What it does	What goes wrong
Locks in short	When a transaction updates a row, it holds a lock on it until the transaction ends.	Running migrations with no lock_timeout.
Find the blocker	Process 2488 waits for a lock held by 2210.	Ignoring deadlock errors instead of fixing lock order and retrying.
Long transactions hurt in other ways	A transaction that stays open holds back the cleanup of old row versions across the database, so tables grow and vacuum cannot remove dead rows.	Leaving transactions open while waiting for user input or HTTP calls.
Deadlocks	A deadlock happens when two transactions each hold a lock the other needs.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Timeouts that protect you	lock_timeout: how long a statement waits for a lock before giving up.	Killing the victim instead of the blocker.

Flow: Locks in short to Find the blocker / Find the blocker to Long transactions hurt in other ways / Long transactions hurt in other ways to Deadlocks / Deadlocks to Timeouts that protect you

Incident: The migration that froze checkout

Incident: *A one-line schema change brought checkout to a halt for four minutes.*

```
$ psql -c "SELECT pid, state, wait_event_type, now()-query_start AS age, left(query,45) FROM pg_st \
    at_activity WHERE state <> 'idle' ORDER BY query_start LIMIT 3;"
 pid  | state  | wait_event_type | age      | left
------+--------+-----------------+----------+---------------------------------------------
 1190 | active |                 | 00:04:10 | SELECT count(*) FROM orders  -- report
 1304 | active | Lock            | 00:03:55 | ALTER TABLE orders ADD COLUMN note text
 1350 | active | Lock            | 00:03:50 | INSERT INTO orders (customer_id, total)
```

A long report held a weak lock on `orders`. The `ALTER TABLE` needed a strong lock and queued behind it. Every new insert then queued behind the `ALTER`, so checkout stopped even though the report only read data. Setting `lock_timeout = '3s'` for the migration lets it fail fast and be retried in a quiet moment, and the report was moved to a replica.

Common mistakes

- Running migrations with no lock_timeout
- Leaving transactions open while waiting for user input or HTTP calls
- Killing the victim instead of the blocker
- Ignoring deadlock errors instead of fixing lock order and retrying

MARGA by EknathaLabs
marga.eknathalabs.com

Check yourself

How long does a row lock last?

Answer: Until the transaction commits or rolls back

What does pg_blocking_pids(pid) return?

Answer: The processes that are blocking the given process

Why is idle in transaction dangerous?

Answer: It holds locks and old row versions while doing nothing

Why set lock_timeout for a schema migration?

Answer: So it fails fast instead of queueing and blocking all other writers

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 4. Connections, pooling and capacity

14 minutes. Understand why too many connections hurt, how a pooler such as PgBouncer helps, and how to size memory and limits.

A database does not slow down smoothly when it is overloaded. It runs fine, then it runs out of connections or memory, and fails quickly. The usual cause is that every application instance opens its own pool of connections.

What you will learn

- Why PostgreSQL connections are expensive
- Connection pool maths for an application fleet
- PgBouncer and its pooling modes
- Memory settings: shared_buffers, work_mem and their traps

Why connections cost

PostgreSQL starts a separate process for each client connection. Each one uses memory and a slot in shared structures, and having hundreds of active sessions makes the CPUs spend time switching between them. A server with 16 cores often performs best with a few dozen running queries, not several hundred.

```
$ psql -c "SHOW max_connections;"
 max_connections
-----------------
 200
$ psql -c "SELECT state, count(*) FROM pg_stat_activity GROUP BY state;"
 state                | count
----------------------+-------
 active               |    14
 idle                 |   171
 idle in transaction  |     9
```

Most of the 194 sessions are idle. They are still using memory and taking slots. When the limit is reached, new clients get:

```
FATAL:  sorry, too many clients already
```

Pool maths

Each application process has its own pool. With 40 pods and a pool of 10 connections each, the application can open 400 connections, which is more than the database allows. Add a second deployment, a cron job and a reporting tool and the limit is crossed with no one noticing, until a deploy restarts all pods at once and every pod reconnects together.

Work out the total: **instances × pool size** across every client, and keep it below `max_connections` minus the reserved slots for administrators (`superuser_reserved_connections`, 3 by default).

PgBouncer

A **pooler** sits between applications and the database. Applications connect to it by the thousands, and it keeps a small number of real connections.

- **Session pooling**: a server connection is assigned for the life of the client connection. Safe, but saves little if clients stay connected.
- **Transaction pooling**: a server connection is assigned only during a transaction. This is the most useful mode, and it limits what the application may use: session-level features such as `SET`, advisory locks, `LISTEN` and (in older versions) prepared statements may not behave as expected.
- **Statement pooling**: one statement at a time. Rarely usable, as it forbids multi-statement transactions.

```
[databases]
shop = host=10.0.2.10 port=5432 dbname=shop
```

MARGA by EknathaLabs
marga.eknathalabs.com

```
[pgbouncer]
pool_mode = transaction
max_client_conn = 2000
default_pool_size = 20
```

With these settings 2 000 clients can connect and at most 20 server connections per database and user pair are used.

Memory settings

- **shared_buffers**: the shared cache. A common starting point is about a quarter of RAM on a dedicated server.
- **work_mem**: memory for **each** sort or hash step in **each** query. It is not per connection but per operation, so a complex query can use several times this value, and 200 connections doing so can add up to many gigabytes. Raising it globally is risky; raise it per role or per session for the reporting job.
- **maintenance_work_mem**: used by vacuum and index builds.
- **effective_cache_size**: tells the planner how much the OS is likely to cache. It allocates nothing.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Connections, pooling and capacity fits together

Part	What it does	What goes wrong
Why connections cost	PostgreSQL starts a separate process for each client connection.	Using transaction pooling with an application that depends on session state.
Pool maths	Each application process has its own pool.	Not leaving reserved connections for administrators.
PgBouncer	A pooler sits between applications and the database.	Raising max_connections instead of fixing the pool sizes.
Memory settings	shared_buffers: the shared cache. A common starting point is about a quarter of RAM on a dedicated server.	Setting work_mem very high globally.

Flow: Why connections cost to Pool maths / Pool maths to PgBouncer / PgBouncer to Memory settings

Incident: Everyone reconnected at once

Incident: *After a deploy the database stopped accepting connections for ten minutes.*

```
FATAL:  sorry, too many clients already
$ psql -c "SELECT count(*) FROM pg_stat_activity;"
 count
-------
   200
$ psql -c "SELECT application_name, count(*) FROM pg_stat_activity GROUP BY 1 ORDER BY 2 DESC LIMI \
    T 2;"
 shop-api     | 170
 shop-worker  |  26
```

The API had 17 pods with a pool of 10 connections each, and a rolling deploy started new pods before closing the old ones, so for a while there were twice as many. The pool fills with failed connection attempts, requests retry, and the load made recovery slower. The team added PgBouncer in transaction mode, set the pool per pod to 5, and limited the surge during deploys.

Common mistakes

- Raising max_connections instead of fixing the pool sizes
- Setting work_mem very high globally

MARGA by EknathaLabs marga.eknathalabs.com

- Using transaction pooling with an application that depends on session state
- Not leaving reserved connections for administrators

Check yourself

Why are many idle PostgreSQL connections a problem?

Answer: Each one uses a process, memory and a connection slot

40 pods each with a pool of 10 connect to a database. How many connections can they open?

Answer: Up to 400

Which PgBouncer mode gives the best reuse of server connections for most web applications?

Answer: Transaction pooling

What is work_mem?

Answer: Memory for each sort or hash step in each query

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 5. Replication lag, vacuum and day-two operations

15 minutes. Monitor replication, understand autovacuum and bloat, and run changes and backups so the database stays healthy.

A database that works on launch day needs care for years: copies must keep up, old rows must be cleaned out, backups must restore, and changes must not hurt users.

What you will learn

- Streaming replication and how to measure lag
- Vacuum, autovacuum and table bloat
- Safe schema changes
- Backups, point-in-time recovery and restore tests

Replication and lag

A **primary** sends its write-ahead log (WAL) to **standbys**, which replay it. Lag is the delay before a change on the primary is visible on a standby. Reading from a standby that is behind can show old data, such as a user's order missing right after it was placed.

```
primary$ psql -c "SELECT client_addr, state, pg_wal_lsn_diff(sent_lsn, replay_lsn) AS bytes_behind \
    , replay_lag FROM pg_stat_replication;"
 client_addr | state     | bytes_behind | replay_lag
-------------+-----------+--------------+-----------------
 10.0.2.21   | streaming |   1572864000 | 00:07:41.28
standby$ psql -c "SELECT now() - pg_last_xact_replay_timestamp() AS delay;"
 delay
-----------------
 00:07:44.1
```

Common causes of lag: a heavy write burst, a slow standby disk, a long query on the standby that conflicts with replay (the setting `max_standby_streaming_delay` decides who gives way), or the network. A **replication slot** keeps WAL for a standby that is offline, and if the standby never returns the primary's disk fills with WAL. Monitor slots and drop abandoned ones.

Vacuum and bloat

When a row is updated or deleted, the old version stays until **vacuum** removes it. **Autovacuum** runs vacuum automatically when about 20 percent of a table (the `autovacuum_vacuum_scale_factor`) plus a small base has changed. On a table with 500 million rows that means 100 million dead rows before it starts, which is far too late. Lower the factor for big, busy tables:

```
ALTER TABLE events SET (autovacuum_vacuum_scale_factor = 0.02);
```

```
$ psql -c "SELECT relname, n_live_tup, n_dead_tup, last_autovacuum FROM pg_stat_user_tables ORDER  \
    BY n_dead_tup DESC LIMIT 2;"
 relname | n_live_tup | n_dead_tup | last_autovacuum
---------+------------+------------+---------------------
 events  |  412000000 |   96000000 | 2026-09-30 03:12:10
```

Vacuum cannot remove a row version that some open transaction might still see, so a long transaction (see the locks lesson) keeps dead rows alive. **Bloat** is the wasted space left behind. `VACUUM` marks it reusable; it does not shrink the file. To shrink a table, use `pg_repack` or `VACUUM FULL` (which locks the table).

A related risk is **transaction ID wraparound**: transaction numbers are finite, and vacuum must freeze old rows. If autovacuum cannot keep up, PostgreSQL eventually refuses writes to protect data. Monitor `age(datfrozenxid)`.

Safe schema changes

MARGA by EknathaLabs marga.eknathalabs.com

- Add columns as nullable with no default or with a constant default (cheap on current versions), then backfill in batches.
- Create indexes with `CONCURRENTLY`.
- Add constraints as `NOT VALID` and validate later, so the lock is short.
- Set `lock_timeout` for the migration, and run it when traffic is low.
- Keep changes backward compatible, so old and new application versions both work during a deploy.

Backups

- **Logical** backups (`pg_dump`) are portable and slow to restore on big databases.
- **Physical** backups with WAL archiving (tools such as pgBackRest) allow **point-in-time recovery**: restore to the minute before the bad change.
- Replication is not a backup: a mistaken `DELETE` replicates to every standby in seconds.
- Decide RPO and RTO, then restore to a scratch server regularly and run a query that checks the data.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Replication lag, vacuum and day-two operations fits together

Part	What it does	What goes wrong
Replication and lag	A primary sends its write-ahead log (WAL) to standbys, which replay it.	Reading important data from a lagging standby.
Vacuum and bloat	When a row is updated or deleted, the old version stays until vacuum removes it.	Leaving autovacuum defaults on very large tables.
Safe schema changes	Add columns as nullable with no default or with a constant default (cheap on current versions), then backfill in batches.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Backups	Logical backups (pg_dump) are portable and slow to restore on big databases.	Believing a replica is a backup.

Flow: Replication and lag to Vacuum and bloat / Vacuum and bloat to Safe schema changes / Safe schema changes to Backups

Incident: The standby that was always behind

Incident: *Customers saw their new orders missing for several minutes, and then the primary ran out of disk.*

```
primary$ psql -c "SELECT slot_name, active, pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), r \
    estart_lsn)) AS retained FROM pg_replication_slots;"
 slot_name    | active | retained
--------------+--------+----------
 reporting_s1 | f      | 412 GB
```

A reporting replica had been switched off a week earlier but its replication slot remained. The primary kept every WAL file since then, 412 GB, until the disk filled. Another standby was lagging because a long report on it conflicted with replay. Dropping the dead slot with `pg_drop_replication_slot` freed the disk, and reports were moved to a replica with its own conflict settings. Replication needs monitoring just as the primary does.

Common mistakes

- Reading important data from a lagging standby
- Leaving autovacuum defaults on very large tables

MARGA by EknathaLabs marga.eknathalabs.com

- Believing a replica is a backup
- Never restoring a backup to see if it works

Check yourself

What can a replication slot with an offline standby cause?

Answer: The primary keeps WAL until its disk fills

Why can autovacuum start too late on a huge table?

Answer: The default scale factor waits for about 20 percent of rows to change

Why is replication not a backup?

Answer: A bad DELETE replicates to the standbys quickly

What does point-in-time recovery need?

Answer: A base backup and archived WAL

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 6. Finding the worst queries with pg_stat_statements

13 minutes. Rank queries by total time, calls and rows with pg_stat_statements, then tune the few that cause most of the load.

Slow-query logs show single bad events, but load comes from patterns. The pg_stat_statements extension groups queries by shape and tells you where the database spends its time, so you can spend yours on the right query.

What you will learn

- Enabling and reading pg_stat_statements
- Ranking by total time, mean time and calls
- Moving from a query to its plan and fix
- Measuring the effect of a change

Enable and read

```
-- postgresql.conf: shared_preload_libraries = 'pg_stat_statements'
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;

SELECT calls,
       round(total_exec_time::numeric, 0) AS total_ms,
       round(mean_exec_time::numeric, 2)  AS mean_ms,
       rows,
       left(query, 80) AS query
FROM pg_stat_statements
ORDER BY total_exec_time DESC
LIMIT 10;
```

Queries are **normalized**, so `WHERE id = 7` and `WHERE id = 9` count as one entry with a placeholder. Sort three ways: by total time for overall load, by mean time for slow single calls and by calls for chatty clients.

From query to fix

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM orders WHERE customer_id = 42 AND status = 'open';

CREATE INDEX CONCURRENTLY orders_customer_status_idx
  ON orders (customer_id, status);
```

Look for sequential scans on large tables, row estimates far from actual rows and many shared buffers read. Typical fixes are a suitable index, selecting only needed columns, removing an unneeded join, batching many small queries into one or caching the result in the application.

Measure

```
SELECT pg_stat_statements_reset();
-- run normal traffic for an hour, then query again
```

Reset the counters before a change and compare after a representative period. Record the before and after total time so the improvement is evidence rather than a feeling.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs
marga.eknathalabs.com

Diagram: How Finding the worst queries with pg_stat_statements fits together

Part	What it does	What goes wrong
Enable and read	Queries are normalized, so WHERE id = 7 and WHERE id = 9 count as one entry with a placeholder.	Tuning only the slowest single query.
From query to fix	Look for sequential scans on large tables, row estimates far from actual rows and many shared buffers read.	Reading plans without ANALYZE and BUFFERS.
Measure	Reset the counters before a change and compare after a representative period.	Not resetting statistics before a test.

Flow: Enable and read to From query to fix / From query to fix to Measure

Incident: The fast query that used all the CPU

Incident: *A cheap query called constantly dominated database load.*

```
calls: 18400000  mean_ms: 4.1  total_ms: 75440000
query: SELECT * FROM sessions WHERE token = $1
Seq Scan on sessions (rows=1 of 2,100,000)
```

The team had chased long reports. Ranking by total time exposed a session lookup with no index on token. One index removed the scan, brought the mean time under 0.1 ms and cut CPU by half.

Common mistakes

- Tuning only the slowest single query
- Not resetting statistics before a test
- Adding indexes without checking write cost
- Reading plans without ANALYZE and BUFFERS

Check yourself

Which ordering shows overall database load best?

Answer: total_exec_time

Why are queries normalized?

Answer: Values become placeholders so shapes group together

What does a large gap between estimated and actual rows suggest?

Answer: Stale or poor statistics

How do you prove a fix worked?

Answer: Compare statistics before and after

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 7. Partitioning and data retention

12 minutes. Split large tables by time or key, drop old data cheaply and plan partitions so maintenance stays fast.

A table with billions of rows can still be fast to query, but it becomes hard to maintain: deletes are slow, vacuum takes forever and indexes grow huge. Partitioning keeps data in manageable pieces and makes retention a metadata operation.

What you will learn

- Range, list and hash partitioning and when to use each
- Creating partitions and automating them
- Dropping and detaching old data safely
- Making queries use partition pruning

Declarative partitioning

```
CREATE TABLE events (
    id bigint GENERATED ALWAYS AS IDENTITY,
    created_at timestamptz NOT NULL,
    kind text NOT NULL,
    payload jsonb
) PARTITION BY RANGE (created_at);

CREATE TABLE events_2026_11 PARTITION OF events
    FOR VALUES FROM ('2026-11-01') TO ('2026-12-01');
CREATE INDEX ON events_2026_11 (created_at, kind);
```

Range partitions by time suit logs and events. **List** partitions split by a category such as region. **Hash** spreads rows evenly across a fixed number of partitions. Primary keys and unique constraints must include the partition key.

Automate and retire

Create next month's partition before it is needed, with a scheduled job or an extension, and alert if it is missing, because inserts outside every range fail. Retire old data by detaching, archiving and dropping:

```
ALTER TABLE events DETACH PARTITION events_2025_11 CONCURRENTLY;
-- archive with pg_dump if required, then:
DROP TABLE events_2025_11;
```

Dropping a partition is fast and does not create dead rows, unlike deleting millions of rows.

Pruning

```
EXPLAIN SELECT count(*) FROM events
WHERE created_at >= '2026-11-01' AND created_at < '2026-11-08';
```

If the plan shows only the relevant partitions, **pruning** works. Queries that omit the partition key scan all partitions. Keep the number of partitions reasonable, because thousands of tiny partitions slow planning.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: How Partitioning and data retention fits together

Part	What it does	What goes wrong
Declarative partitioning	Range partitions by time suit logs and events.	Forgetting to create future partitions.
Automate and retire	Create next month's partition before it is needed, with a scheduled job or an extension, and alert if it is missing, because inserts outside every range fail.	Deleting rows instead of dropping partitions.
Pruning	If the plan shows only the relevant partitions, pruning works.	Queries that do not use the partition key.

Flow: Declarative partitioning to Automate and retire / Automate and retire to Pruning

Incident: No partition for today

Incident: *Inserts failed at midnight because the next partition did not exist.*

```
ERROR:  no partition of relation "events" found for row
DETAIL:  Partition key of the failing row contains (created_at) = (2026-12-01 00:00:03+00).
```

The job that created monthly partitions had been failing silently for weeks. At the month boundary writes began to fail. The team added a default partition as a safety net, alerts for missing future partitions and a test that checks three months ahead.

Common mistakes

- Forgetting to create future partitions
- Deleting rows instead of dropping partitions
- Queries that do not use the partition key
- Creating thousands of tiny partitions

Check yourself

What is the main retention benefit of partitioning?

Answer: Old data can be dropped quickly

What must a primary key include?

Answer: The partition key

What is partition pruning?

Answer: The planner skips partitions that cannot match

What protects against a missing partition?

Answer: A default partition and alerts

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 8. Backup and point-in-time recovery operations for SQL

13 minutes. Run base backups and WAL archiving, perform a point-in-time restore and test restore time against your recovery objectives.

Point-in-time recovery lets you undo mistakes, not just survive disk loss. It needs a base backup, a continuous stream of write-ahead log files and a practiced procedure. This lesson shows the moving parts and how to test them.

What you will learn

- How base backups and WAL archiving work together
- Restoring to a specific time on a new server
- Measuring RPO and RTO
- Monitoring and testing the backup system

The pieces

A **base backup** is a full copy of the data directory at one moment. **WAL archiving** copies every write-ahead log segment to safe storage. To recover, restore the base backup and replay WAL up to a target time, transaction or label.

```
# with pgBackRest
pgbackrest --stanza=main backup --type=full
pgbackrest --stanza=main info

# postgresql.conf
archive_mode = on
archive_command = 'pgbackrest --stanza=main archive-push %p'
```

Restore to a point in time

```
# on a NEW server, stopped PostgreSQL
pgbackrest --stanza=main --type=time \
  --target="2026-11-05 14:02:00+00" \
  --target-action=promote restore
systemctl start postgresql
psql -c "SELECT count(*) FROM invoices;"
```

Never restore over the live server. Restore alongside, verify the data, then export what you need or switch traffic after careful checks.

RPO and RTO

RPO is how much data you can lose, bounded by how far behind WAL archiving can lag. **RTO** is how long recovery takes, driven by backup size, network speed and replay time. Measure both in a real restore, and write the numbers in the runbook.

Keep it healthy

Alert when `pg_stat_archiver` shows failures or when the newest archived WAL is old. Store backups in a different failure domain, encrypt them, keep retention long enough to catch slow-moving mistakes and run a full restore test every month.

```
SELECT last_archived_time, failed_count, last_failed_time FROM pg_stat_archiver;
```

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: How Backup and point-in-time recovery operations for... fits together

Part	What it does	What goes wrong
The pieces	A base backup is a full copy of the data directory at one moment.	Never testing a full restore.
Restore to a point in time	Never restore over the live server. Restore alongside, verify the data, then export what you need or switch traffic after careful checks.	Restoring over the live server.
RPO and RTO	RPO is how much data you can lose, bounded by how far behind WAL archiving can lag.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Keep it healthy	Alert when pg_stat_archiver shows failures or when the newest archived WAL is old.	Backups stored on the same failure domain.

Flow: The pieces to Restore to a point in time / Restore to a point in time to RPO and RTO / RPO and RTO to Keep it healthy

Incident: Archiving had been failing for nine days

Incident: *A restore needed WAL that was never archived.*

```
FATAL:  could not locate required checkpoint record
pg_stat_archiver: failed_count = 11204
last_failed_time: 9 days ago
```

A full disk on the archive target had stopped copies, but the base backup job still reported success. When recovery was needed the log chain was broken. The team alerts on archiver failures and lag, monitors the target and runs a monthly restore test that must reach a given time.

Common mistakes

- Never testing a full restore
- Restoring over the live server
- Backups stored on the same failure domain
- Ignoring WAL archive failures

Check yourself

What does PITR need besides a base backup?

Answer: Archived WAL

What does RPO measure?

Answer: How much data you can lose

Where should you restore for a test?

Answer: A separate new server

What should you alert on?

Answer: Archive failures and lag

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 9. Online schema changes without downtime

13 minutes. Change tables on a busy database safely with lock timeouts, concurrent indexes, backfills in batches and the expand and contract pattern.

Schema changes are routine but dangerous on a live system. The risk is rarely the work itself; it is the lock the statement needs and the queue of queries that forms behind it. A handful of habits make changes safe.

What you will learn

- Which operations take strong locks
- Using lock_timeout and CONCURRENTLY
- Backfilling data in batches
- Adding constraints without blocking writes

Locks and timeouts

```
SET lock_timeout = '3s';
SET statement_timeout = '15min';

ALTER TABLE orders ADD COLUMN channel text;           -- fast, no rewrite
CREATE INDEX CONCURRENTLY orders_channel_idx ON orders (channel);
```

`ALTER TABLE` needs a strong lock briefly, but if a long transaction holds a conflicting lock it waits, and every later query waits behind it. A **lock_timeout** makes the migration fail fast so you can retry in a quieter moment. `CREATE INDEX CONCURRENTLY` avoids blocking writes but takes longer and cannot run inside a transaction.

Backfill in batches

```
-- repeat until 0 rows are updated
UPDATE orders SET channel = 'web'
WHERE id IN (SELECT id FROM orders WHERE channel IS NULL LIMIT 5000);
```

One huge update holds locks, bloats the table and creates replication lag. Small batches with a short pause let vacuum and replicas keep up. Watch lag while it runs.

Constraints

```
ALTER TABLE orders ADD CONSTRAINT orders_channel_chk
  CHECK (channel IS NOT NULL) NOT VALID;
ALTER TABLE orders VALIDATE CONSTRAINT orders_channel_chk;
```

Adding a constraint as `NOT VALID` checks only new rows and needs a short lock. `VALIDATE` then scans existing rows with a weaker lock. The same idea applies to foreign keys.

Expand and contract

Make changes compatible in steps: **expand** by adding the new column, deploy code that writes both, backfill, switch reads, then **contract** by removing the old column in a later release. Each step can be rolled back.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: How Online schema changes without downtime fits together

Part	What it does	What goes wrong
Locks and timeouts	ALTER TABLE needs a strong lock briefly, but if a long transaction holds a conflicting lock it waits, and every later query waits behind it.	Changing hot tables without lock_timeout.
Backfill in batches	One huge update holds locks, bloats the table and creates replication lag.	One giant backfill update.
Constraints	Adding a constraint as NOT VALID checks only new rows and needs a short lock.	Adding constraints with full validation inline.
Expand and contract	Make changes compatible in steps: expand by adding the new column, deploy code that writes both, backfill, switch reads, then contract by removing the old column in a...	Dropping a column in the same release as the code change.

Flow: Locks and timeouts to Backfill in batches / Backfill in batches to Constraints / Constraints to Expand and contract

Incident: The index that blocked all writes

Incident: *A plain CREATE INDEX stalled writes on a hot table.*

```
CREATE INDEX orders_status_idx ON orders (status);
waiting for ShareLock on relation orders
412 queries waiting, longest 241s
```

A plain index build takes a lock that blocks writes for the whole build. On a large table that was minutes of failed requests. The team standardized on CONCURRENTLY with a lock timeout, added a migration review checklist and ran a rehearsal against a copy of production.

Common mistakes

- Changing hot tables without lock_timeout
- One giant backfill update
- Adding constraints with full validation inline
- Dropping a column in the same release as the code change

Check yourself

What does CONCURRENTLY do for index builds?

Answer: Avoids blocking writes

Why use lock_timeout?

Answer: To fail fast instead of blocking queries

What is NOT VALID for constraints?

Answer: Checks only new rows until validated

What is the contract step?

Answer: Removing the old structure after the switch

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 10. Replicas, switchover and failover operations

13 minutes. Run streaming replicas, tell planned switchover from emergency failover, avoid split brain and handle replication lag.

Replicas give you read capacity and a way to survive a lost server. The hard part is the moment of change: promoting a replica safely without two servers believing they are the primary.

What you will learn

- Streaming replication and measuring lag
- Planned switchover versus emergency failover
- Split brain and fencing
- Rebuilding a failed primary

Replication and lag

```
-- on the primary
SELECT client_addr, state, sent_lsn, replay_lsn,
       pg_wal_lsn_diff(sent_lsn, replay_lsn) AS lag_bytes
FROM pg_stat_replication;

-- on a replica
SELECT now() - pg_last_xact_replay_timestamp() AS replay_delay;
```

Lag grows when replicas are slow, the network is busy or a long query on the replica conflicts with replay. Alert on both bytes and seconds. Reads from a lagging replica can return old data, so route read-your-own-write traffic to the primary.

Switchover and failover

A **switchover** is planned: stop writes, let the replica catch up fully, promote it and demote the old primary with no data loss. A **failover** is unplanned: the primary is gone, you promote the most up-to-date replica and accept that some recent transactions may be lost if replication was asynchronous.

```
patronictl -c /etc/patroni.yml list
patronictl -c /etc/patroni.yml switchover --candidate db2 --scheduled now
```

Synchronous replication reduces data loss at the price of write latency and availability if the replica is down.

Avoiding split brain

Use a **consensus** store and **fencing**: the old primary must be unable to accept writes before another is promoted, for example by losing its leader lock, being shut down or being cut off at the network layer. Clients should connect through a virtual address, proxy or DNS name that follows the leader.

Rebuild

An old primary may have extra transactions. Rewind it with `pg_rewind` or rebuild it from a base backup, then join it as a replica. Never simply restart it as a primary.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: How Replicas, switchover and failover operations fits together

Part	What it does	What goes wrong
Replication and lag	Lag grows when replicas are slow, the network is busy or a long query on the replica conflicts with replay.	Ignoring replica lag in application reads.
Switchover and failover	A switchover is planned: stop writes, let the replica catch up fully, promote it and demote the old primary with no data loss.	Automatic failover that was never tested.
Avoiding split brain	Use a consensus store and fencing: the old primary must be unable to accept writes before another is promoted, for example by losing its leader lock, being shut down or...	Promoting without fencing the old primary.
Rebuild	An old primary may have extra transactions.	Restarting a failed primary as a primary.

Flow: Replication and lag to Switchover and failover / Switchover and failover to Avoiding split brain / Avoiding split brain to Rebuild

Incident: Two primaries

Incident: *A partitioned old primary kept accepting writes after promotion.*

```
db1: still accepting writes (leader lock lost 4m ago)
db2: promoted to primary
diverged writes: 1,842 rows on db1 only
```

The failover manager promoted db2 but nothing stopped db1 from running. Some application servers kept writing to it. The team enabled a watchdog so a node without the leader lock shuts down its database, moved clients behind a proxy that follows the leader, and wrote a runbook for reconciling data if it happens again.

Common mistakes

- Promoting without fencing the old primary
- Restarting a failed primary as a primary
- Ignoring replica lag in application reads
- Automatic failover that was never tested

Check yourself

What is a switchover?

Answer: A planned role change with no data loss

What is split brain?

Answer: Two servers both accepting writes as primary

How is split brain prevented?

Answer: Consensus and fencing

What can a lagging replica return?

Answer: Stale data

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 11. Autovacuum, bloat and maintenance windows

12 minutes. Keep tables healthy by tuning autovacuum, measuring bloat, avoiding transaction ID wraparound and planning safe maintenance.

Updates and deletes in PostgreSQL create new row versions and leave old ones behind. Autovacuum reclaims them in the background. When it cannot keep up, or cannot remove rows because something still needs them, tables bloat and performance drops.

What you will learn

- Dead tuples, bloat and what autovacuum does
- Finding what blocks vacuum
- Tuning autovacuum for busy tables
- Transaction ID wraparound and safe maintenance

Watch vacuum

```
SELECT relname, n_live_tup, n_dead_tup,
       last_autovacuum, last_autoanalyze
FROM pg_stat_user_tables
ORDER BY n_dead_tup DESC LIMIT 10;
```

A growing `n_dead_tup` with an old `last_autovacuum` means vacuum is behind. Track table and index size over time, because steady growth with a constant row count is bloat.

What blocks cleanup

Vacuum cannot remove a row version that some transaction might still see.

```
SELECT pid, now() - xact_start AS age, state, left(query, 60)
FROM pg_stat_activity
WHERE xact_start IS NOT NULL ORDER BY age DESC LIMIT 5;

SELECT slot_name, active, restart_lsn FROM pg_replication_slots;
```

Long transactions, idle-in-transaction sessions and inactive replication slots are the usual causes. Fix them first, then tune.

Tuning

```
ALTER TABLE events SET (
  autovacuum_vacuum_scale_factor = 0.02,
  autovacuum_analyze_scale_factor = 0.01,
  autovacuum_vacuum_cost_limit = 2000
);
```

The default triggers at 20 percent dead rows, which is too late for huge tables. Lower the scale factor per hot table and raise the cost limit so vacuum works faster.

Wraparound and maintenance

Transaction IDs are finite. Monitor `age(datfrozenxid)` and keep it far from the limit. To shrink an already bloated table, use `pg_repack` or plan a `VACUUM FULL`, which locks the table, in a maintenance window with a rollback plan.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs
marga.eknathalabs.com

Diagram: How Autovacuum, bloat and maintenance windows fits together

Part	What it does	What goes wrong
Watch vacuum	A growing n_dead_tup with an old last_autovacuum means vacuum is behind.	Only raising autovacuum frequency when something blocks it.
What blocks cleanup	Vacuum cannot remove a row version that some transaction might still see.	Forgetting inactive replication slots.
Tuning	The default triggers at 20 percent dead rows, which is too late for huge tables.	Running VACUUM FULL during business hours.
Wraparound and maintenance	Transaction IDs are finite. Monitor age(datfrozenxid) and keep it far from the limit.	Ignoring transaction ID age.

Flow: Watch vacuum to What blocks cleanup / What blocks cleanup to Tuning / Tuning to Wraparound and maintenance

Incident: The transaction that never ended

Incident: *An idle transaction blocked cleanup for three weeks.*

```
pid 8841  idle in transaction  xact_start: 21 days ago
n_dead_tup: 184220001  last_autovacuum: 2 minutes ago
table size: 412 GB (live data about 40 GB)
```

A script opened a transaction and never closed it. Autovacuum ran constantly but could not remove old rows. After the session was ended the team added idle_in_transaction_session_timeout and an alert on transactions older than 15 minutes.

Common mistakes

- Only raising autovacuum frequency when something blocks it
- Forgetting inactive replication slots
- Running VACUUM FULL during business hours
- Ignoring transaction ID age

Check yourself

What blocks vacuum from removing rows?

Answer: Old transactions or replication slots

What is bloat?

Answer: Space taken by dead row versions

What setting stops forgotten transactions?

Answer: idle_in_transaction_session_timeout

What does VACUUM FULL do?

Answer: Rewrites the table with a strong lock

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 12. A database incident runbook

12 minutes. Triage a database incident in order: confirm impact, check locks, connections, disk and replication, mitigate safely and write down what happened.

During an incident people guess. A written order of checks removes guessing, shortens the time to a diagnosis and leaves evidence for the review afterwards.

What you will learn

- The order of checks: scope, connections, locks, resources, replication, changes
- Queries that reveal each cause
- Choosing the smallest safe mitigation
- Capturing evidence for the postmortem

Runbook order

1. **Scope:** who is affected, since when, which endpoints
2. **Connections:** are we at the limit or waiting on a pool
3. **Long queries and locks:** who blocks whom
4. **Resources:** CPU, memory, disk space and I/O
5. **Replication:** lag, broken replica, stuck slot
6. **Changes:** deploys, migrations, config or traffic changes

```
-- connections by state
SELECT state, count(*) FROM pg_stat_activity GROUP BY 1 ORDER BY 2 DESC;

-- longest running queries
SELECT pid, now() - query_start AS runtime, wait_event_type, left(query, 70)
FROM pg_stat_activity WHERE state <> 'idle' ORDER BY runtime DESC LIMIT 5;

-- who blocks whom
SELECT pid, pg_blocking_pids(pid) AS blocked_by, left(query, 60)
FROM pg_stat_activity WHERE cardinality(pg_blocking_pids(pid)) > 0;

df -h /var/lib/postgresql
iostat -x 1 3
```

Mitigate

Pick the smallest safe action: cancel a runaway query with `pg_cancel_backend`, terminate a blocker with `pg_terminate_backend`, pause a job, shed load or fail over if the server is truly unhealthy. A full disk may need removing old logs or WAL carefully, never deleting files in the data directory by hand. Restarting loses caches and hides evidence, so it is rarely the first step.

Evidence

Before changing anything, save the output of the queries above with a timestamp. Note each action and its effect. This record turns a stressful hour into a clear timeline for the review.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: How A database incident runbook fits together

Part	What it does	What goes wrong
Runbook order	Scope: who is affected, since when, which endpoints.	Deleting WAL or data files to free space.
Mitigate	Pick the smallest safe action: cancel a runaway query with pg_cancel_backend, terminate a blocker with pg_terminate_backend, pause a job, shed load or fail over if the...	Restarting before collecting evidence.
Evidence	Before changing anything, save the output of the queries above with a timestamp.	Killing queries without finding the blocker.

Flow: Runbook order to Mitigate / Mitigate to Evidence

Incident: Afternoon latency spikes

Incident: *A report query held locks and used all connections each afternoon.*

```
active: 96 of 100 connections
pid 7314  runtime 00:41:12  report_daily_totals
54 queries blocked by pid 7314
```

An analyst's report ran in a transaction with an update, blocking the application. The on-call engineer cancelled it, saved the output, and the team moved reporting to a replica with a statement timeout and a lower connection limit for the reporting role.

Common mistakes

- Restarting before collecting evidence
- Deleting WAL or data files to free space
- Killing queries without finding the blocker
- Skipping the review afterwards

Check yourself

What is the first runbook step?

Answer: Confirm scope and impact

Which function lists blockers?

Answer: pg_blocking_pids

What is a safer first action than a restart?

Answer: Cancel or terminate the blocking query

Why save evidence first?

Answer: It supports the postmortem

MARGA by EknathaLabs
marga.eknathalabs.com

Cheat sheet: SQL performance and database operations

Plans, indexes, locks, connections, replication and vacuum, with the queries that show what a busy PostgreSQL database is doing.

Find slow queries

```
SELECT calls, round(mean_exec_time) AS mean_ms, round(total_exec_time) AS total_ms, left(query, 80 \
    )
FROM pg_stat_statements ORDER BY total_exec_time DESC LIMIT 10;

EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM orders WHERE customer_id = 42;
```

Look for sequential scans on big tables, estimated rows far from actual rows, and sorts that spill to disk.

Indexes

```
CREATE INDEX CONCURRENTLY orders_cust_status ON orders (customer_id, status);
SELECT relname, indexrelname, idx_scan FROM pg_stat_user_indexes ORDER BY idx_scan LIMIT 10;
ANALYZE orders;
```

Warning: *A plain CREATE INDEX blocks writes. Use CONCURRENTLY on live tables.*

Sessions and locks

```
SELECT pid, state, now()-xact_start AS tx_age, wait_event_type, left(query, 60)
FROM pg_stat_activity WHERE state <> 'idle' ORDER BY tx_age DESC NULLS LAST;

SELECT pid, pg_blocking_pids(pid) AS blocked_by FROM pg_stat_activity WHERE cardinality(pg_blockin \
    g_pids(pid)) > 0;
SELECT pg_cancel_backend(2214);      -- cancel query
SELECT pg_terminate_backend(2214);   -- end session
```

Connections

```
SELECT state, count(*) FROM pg_stat_activity GROUP BY state;
SHOW max_connections;
```

Use a pooler. Set `idle_in_transaction_session_timeout`.

Vacuum and bloat

```
SELECT relname, n_live_tup, n_dead_tup, last_autovacuum FROM pg_stat_user_tables ORDER BY n_dead_t \
    up DESC LIMIT 10;
VACUUM (VERBOSE, ANALYZE) orders;
```

An old open transaction stops vacuum from removing dead rows.

Replication

```
SELECT now() - pg_last_xact_replay_timestamp() AS replica_lag;   -- on the replica
SELECT client_addr, state, sent_lsn, replay_lsn FROM pg_stat_replication;  -- on the primary
```

Quick diagnosis

Symptom	First look
One query slow	EXPLAIN ANALYZE, indexes, statistics
Everything slow	top statements, I/O, locks, connections
Too many connections	state grouping, idle in transaction, pooler
Table keeps growing	dead tuples, vacuum, old transactions
Replica behind	replay blocked, I/O, batch size

Cautions

Danger: `kill` *of a long transaction or* `pg_terminate_backend` *rolls it back. Check what it is doing first.*

MARGA by EknathaLabs
marga.eknathalabs.com

Warning: *A plain* `CREATE INDEX` *blocks writes on the table. Use* `CREATE INDEX CONCURRENTLY` *on live systems.*

Warning: *Raising* `max_connections` *does not fix a connection storm and can exhaust memory. Add a pooler.*

Caution: `VACUUM FULL` *locks the table and rewrites it. Use normal vacuum, or a repack tool, on busy tables.*

MARGA by EknathaLabs
marga.eknathalabs.com